

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к индивидуальному домашнему заданию
по дисциплине
«Введение в нереляционные системы управления базами данных»
Тема: Генеалогическое дерево династии Романовых

Студент(ы) гр. 3341

_____ Мильхерт А.С.

_____ Моисеева А.Е.

Студентка гр. 3342

_____ Епонишникова А.И.

Преподаватель

_____ Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студент(ы)

Мильхерт А.С.

Моисеева А.Е.

Епонишникова А.И.

Группа 3341 и 3341

Тема проекта: Генеалогическое дерево династии Романовых.

Исходные данные:

Необходимо реализовать веб-приложение для хранения, визуализации и анализа генеалогического дерева династии Романовых с использованием графовой СУБД Neo4j.

Содержание пояснительной записки:

«Содержание»

«Введение»

«Качественные требования к решению»

«Сценарий использования»

«Модель данных»

«Разработка приложения»

«Вывод»

«Приложение»

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания:

Дата сдачи реферата:

Дата защиты реферата:

Студент(ы) гр. 3341

_____ Мильхерт А.С.

_____ Моисеева А.Е.

Студентка гр. 3342

_____ Епонишникова А.И.

Преподаватель

_____ Заславский М.М.

АННОТАЦИЯ

В рамках данного курса предполагалось разработать веб-приложение в команде на одну из поставленных тем с использованием нереляционной СУБД. Была выбрана тема создания приложения для хранения, визуализации и анализа генеалогического дерева династии Романовых с использованием графовой СУБД Neo4j, так как генеалогические данные имеют внутренне присущую графовую структуру и являются отличным примером для демонстрации преимуществ графовых баз данных перед реляционными. Приложение позволяет выполнять CRUD-операции над персонами и их связями, осуществлять поиск и фильтрацию, визуализировать родословное дерево в виде интерактивного графа, а также импортировать и экспортировать данные в форматах JSON, CSV и XML. Найти исходный код и всю дополнительную информацию можно по ссылке: <https://github.com/moevm/nsql1h26-tree>

ANNOTATION

As part of this course, the team was tasked with developing a web application on one of the proposed topics using a non-relational DBMS. The topic of creating an application for storing, visualizing and analyzing the genealogical tree of the Romanov dynasty using the Neo4j graph database was chosen, since genealogical data has an inherently graph-like structure and serves as an excellent example for demonstrating the advantages of graph databases over relational ones. The application supports CRUD operations on persons and their relationships, search and filtering, interactive graph visualization of the family tree, as well as data import and export in JSON, CSV and XML formats. Source code and all the additional information can be found at: <https://github.com/moevm/nsql1h26-tree>

СОДЕРЖАНИЕ

Введение	7
1. Постановка задачи	9
1.1. Предлагаемое решение	9
1.2. Качественные требования к решению	9
2. Сценарии использования	11
2.1. Макет UI	18
3. Модель данных	19
3.1. Нереляционная модель данных (Neo4j)	19
3.2. Аналог модели данных для SQL СУБД	21
3.3. Сравнение моделей	24
4. Разработанное приложение	26
4.1. Краткое описание	26
4.2. Используемые технологии	26
4.3. Снимки экрана приложения	27
5. Выводы	28
5.1. Достигнутые результаты	28
5.2. Недостатки и пути для улучшения	28
5.3. Будущее развитие решения	29
Приложение А. Документация по сборке и развертыванию	30
Приложение Б. Инструкция для пользователя	31
Литература	36

Введение

Генеалогические данные представляют собой один из наиболее характерных примеров информации, имеющей графовую природу. Родословные деревья, семейные связи и династические отношения естественным образом описываются в терминах узлов (персоны) и ребер (родственные связи между ними). Традиционные реляционные системы управления базами данных (РСУБД) требуют создания множества таблиц и сложных операций соединения (JOIN) для представления и обхода таких структур, что приводит к снижению производительности при глубоком обходе дерева и усложнению запросов.

Графовые базы данных, напротив, хранят связи как объекты первого класса, что позволяет выполнять обход графа за константное время на каждый переход, независимо от общего объема данных. СУБД Neo4j, являющаяся одной из наиболее зрелых и широко используемых графовых баз данных, предоставляет декларативный язык запросов Cypher, специально разработанный для описания паттернов в графовых данных. Запросы на Cypher для поиска предков, потомков и родственных связей оказываются значительно более компактными и читаемыми по сравнению с эквивалентными SQL-запросами с рекурсивными CTE.

Династия Романовых, правившая Россией с 1613 по 1917 год, представляет собой обширное генеалогическое дерево, включающее сотни исторических персон и тысячи родственных связей на протяжении более чем трех столетий. Хранение, визуализация и анализ данных о династии Романовых является практической задачей, хорошо подходящей для демонстрации преимуществ графовых СУБД перед реляционными при работе с иерархическими и сетевыми данными.

Целью настоящей работы является разработка веб-приложения для хранения, визуализации и анализа генеалогического дерева династии

Романовых с использованием графовой СУБД Neo4j. Приложение должно обеспечивать полный цикл работы с данными: импорт и экспорт в машиночитаемых форматах (JSON, CSV, XML), ручное добавление и редактирование персон, поиск и фильтрацию по различным критериям, визуализацию родословного дерева в виде интерактивного графа, а также формирование статистической информации.

В качестве технологического стека выбраны: Neo4j Community Edition в качестве СУБД, Python с фреймворком FastAPI для реализации REST API серверной части, React с инструментом сборки Vite для клиентской части. Все компоненты приложения контейнеризированы с помощью Docker и оркестрируются через Docker Compose, что обеспечивает простоту развертывания и воспроизводимость среды.

В настоящей пояснительной записке описаны постановка задачи, сценарии использования, модель данных для графовой и реляционной СУБД с их сравнительным анализом, архитектура и технологии разработанного приложения, а также выводы по результатам работы.

1. Постановка задачи

Необходимо разработать веб-приложение для хранения, визуализации и анализа генеалогического дерева династии Романовых с использованием нереляционной (графовой) СУБД Neo4j.

1.1. Предлагаемое решение

Предлагаемое решение представляет собой трехзвенное веб-приложение, состоящее из следующих компонентов:

1) Графовая база данных Neo4j 5.18 Community Edition для хранения персон и родственных связей между ними в виде узлов и ребер графа.

2) Серверная часть (backend) на языке Python 3.12 с использованием фреймворка FastAPI, предоставляющая REST API для всех операций над данными.

3) Клиентская часть (frontend) в виде одностраничного приложения (SPA) на React 19 с инструментом сборки Vite 8, обеспечивающая пользовательский интерфейс.

Все компоненты контейнеризированы с помощью Docker и управляются через Docker Compose, что позволяет развернуть приложение единой командой.

1.2. Качественные требования к решению

К разрабатываемому приложению предъявляются следующие качественные требования:

Удобство использования: интуитивно понятный интерфейс с навигационным меню, интерактивный граф с возможностью масштабирования и перемещения, модальные окна для просмотра информации о персонах без перехода между страницами.

Контейнеризация: все компоненты (база данных, backend, frontend) должны запускаться через Docker Compose единой командой без необходимости ручной установки зависимостей.

Импорт и экспорт данных: поддержка трех форматов обмена данными (JSON, CSV, XML) для обеспечения совместимости с различными внешними системами.

Целостность данных: валидация входных данных на стороне клиента и сервера, каскадное удаление связей при удалении персоны, предзаполнение базы данных начальным набором исторических данных.

2. Сценарии использования

В системе определена единственная роль: Пользователь. Ниже описаны основные сценарии использования приложения.

UC-01. Просмотр главной страницы

Роль: Пользователь.

Предусловие: приложение запущено, в БД есть данные.

Основной поток:

1. Пользователь открывает приложение.
2. Система отображает страницу династии Романовых со статистикой: количество персон, связей, средний возраст, период существования династии.
3. Система отображает список последних добавленных персон.
4. Пользователь переходит к любому разделу через навигационное меню или кнопки быстрых действий.

Результат: пользователь видит общую сводку по династии Романовых.

UC-02. Поиск и фильтрация персон

Роль: Пользователь.

Предусловие: в БД есть персоны.

Основной поток:

1. Пользователь переходит на страницу “Поиск”.
2. Пользователь заполняет один или несколько фильтров: имя (подстрока, регистронезависимо), фамилия, титул, пол (выбор из списка), год рождения от/до, год смерти от/до.
3. Пользователь нажимает “Найти”.
4. Система отображает таблицу результатов.
5. Пользователь нажимает на найденную персону в строке таблицы и переходит на карточку (UC-03).

Альтернативный поток: ни одна персона не найдена -- система отображает сообщение “Ничего не найдено”.

Результат: пользователь видит отфильтрованный список персон.

UC-03. Просмотр карточки персоны

Роль: Пользователь.

Предусловие: персона существует в БД.

Основной поток:

1. Пользователь кликает на узел в графе или на строку персоны в таблице поиска (UC-02).

2. Система открывает модальное окно поверх графа.

3. Модальное окно отображает атрибуты персоны: полное имя, год рождения, год смерти, титул, страна, династия, дата добавления, дата изменения, поле комментария.

4. Модальное окно отображает таблицу родственных связей: тип связи, имя персоны, годы жизни.

5. Пользователь кликает на имя связанной персоны в таблице -- система открывает её карточку.

6. Пользователь закрывает модальное окно кнопкой “х” -- возвращается к графу.

Результат: пользователь видит полную информацию о персоне, не покидая страницу графа.

UC-04. Просмотр графа дерева Романовых

Роль: Пользователь.

Предусловие: в БД есть персоны и связи.

Основной поток:

1. Пользователь переходит на страницу “Граф дерева”.

2. Система отображает интерактивный граф всей династии: узлы -- персоны, рёбра сплошные -- связь родитель-ребёнок, рёбра красные -- связь супруги.

3. Пользователь использует кнопки зума (+/-) и центрирования для навигации по графу.

4. Пользователь кликает на узел -- система открывает карточку персоны (UC-03).

Результат: пользователь видит визуализацию родословного дерева.

UC-05. Просмотр кастомизируемой статистики

Роль: Пользователь.

Предусловие: в БД есть персоны.

Основной поток:

1. Пользователь переходит на страницу “Статистика”.

2. Пользователь задаёт фильтры подмножества данных: страна, пол, период жизни от/до.

3. Пользователь выбирает ось X: год рождения / титул / страна.

4. Пользователь выбирает ось Y: количество персон / средняя продолжительность жизни / количество браков.

5. Пользователь нажимает “Построить”.

6. Система сортирует данные на сервере и отображает график с подписями осей.

Примечание: в текущей версии приложения страница статистики существует в виде заглушки и не полностью реализована.

Результат: пользователь видит диаграмму по выбранным параметрам.

UC-06. Экспорт всех данных

Роль: Пользователь.

Предусловие: в БД есть данные.

Основной поток:

1. Пользователь переходит на страницу “Импорт/Экспорт”.
2. Пользователь выбирает формат файла: JSON, CSV или XML.
3. Пользователь нажимает “Экспортировать”.
4. Система формирует файл, содержащий все персоны и все связи.
5. Файл скачивается в браузер.

Результат: пользователь получает файл с полной копией данных приложения.

UC-07. Импорт данных

Роль: Пользователь.

Предусловие: у пользователя есть файл в формате JSON, CSV или XML.

Основной поток:

1. Пользователь переходит на страницу “Импорт/Экспорт”.
2. Пользователь выбирает файл через кнопку “Выбрать файл”.
3. Система проверяет формат файла.
4. Система полностью заменяет текущее содержимое БД данными из файла.
5. Система отображает результат: количество добавленных персон, связей, статус операции.

Альтернативный поток: файл не прошёл валидацию -- система выводит сообщение об ошибке, данные в БД не изменяются.

Результат: содержимое БД полностью заменено данными из файла.

UC-08. Добавление новой персоны

Роль: Пользователь.

Предусловие: приложение запущено.

Основной поток:

1. Пользователь нажимает “Добавить персону” в навигации.
2. Система открывает экран с пустой формой.
3. Пользователь заполняет поля: имя (обязательное), фамилия (обязательное), год рождения, год смерти, титул, страна, династия, комментарий.
4. Пользователь добавляет связи: нажимает “+ Добавить связь”, выбирает тип (отец/мать/сын/дочь/супруг), находит персону через поиск по имени.
5. Пользователь нажимает “Сохранить”.
6. Система создаёт узел и указанные рёбра в БД.
7. Система отображает уведомление об успешном добавлении.

Альтернативный поток: обязательные поля не заполнены -- система выделяет их, сохранение не происходит.

Результат: новая персона добавлена в дерево со всеми указанными связями.

UC-09. Редактирование персоны

Роль: Пользователь.

Предусловие: персона существует в БД, карточка персоны открыта.

Основной поток:

1. Пользователь нажимает “Редактировать” в карточке персоны.
2. Система открывает экран с предзаполненной формой.
3. Пользователь изменяет нужные поля.
4. Пользователь добавляет новые связи или удаляет существующие через таблицу связей.
5. Пользователь нажимает “Сохранить”.
6. Система обновляет данные в БД, фиксирует дату изменения.

Альтернативный поток: пользователь нажимает “Отмена” -- изменения не сохраняются.

Результат: данные персоны и/или её связи обновлены.

UC-10. Удаление персоны

Роль: Пользователь.

Предусловие: персона существует в БД, карточка персоны открыта.

Основной поток:

1. Пользователь нажимает “Удалить” в карточке персоны.
2. Система отображает диалог подтверждения.
3. Пользователь подтверждает удаление.
4. Система удаляет узел и все рёбра персоны из БД.
5. Система закрывает модальное окно и обновляет граф.

Альтернативный поток: пользователь отменяет -- удаление не происходит.

Результат: персона и все её связи удалены из дерева.

Сценарии импорта данных

Массовый импорт данных осуществляется через страницу “Импорт/Экспорт” (UC-07). Пользователь загружает файл в формате JSON, CSV или XML, соответствующий схеме приложения. При импорте текущее содержимое базы данных полностью заменяется данными из файла. Система производит валидацию формата и структуры файла перед импортом.

Ручной ввод данных осуществляется через форму добавления персоны (UC-08). Пользователь заполняет атрибуты персоны и указывает родственные связи с уже существующими в базе персонами. Поиск связанных персон производится через встроенное поле поиска с автодополнением.

Сценарии представления данных

Визуализация в виде графа (UC-04): интерактивный граф отображает все персоны как узлы и все связи как ребра. Узлы раскрашены по полу (фиолетовый для мужчин, розовый для женщин). Сплошные линии обозначают связь родитель-ребёнок, красные линии -- брачные связи. Граф поддерживает масштабирование колесом мыши и перетаскивание.

Таблица результатов поиска (UC-02): отображает отфильтрованный список персон с полями имя, фамилия, титул, годы жизни, пол. Клик по строке открывает карточку персоны.

Карточка персоны (UC-03): модальное окно с полной информацией о персоне и таблицей родственных связей. Позволяет перемещаться между связанными персонами без закрытия модального окна.

Главная страница (UC-01): отображает общую статистику (количество персон, средний возраст, период существования династии) и список последних добавленных персон.

Сценарии анализа данных

На главной странице отображается базовая статистика: общее количество персон в базе, количество связей, средний возраст (рассчитывается как среднее значение разности года смерти и года рождения для персон, у которых известны обе даты), а также период существования династии.

Страница кастомизируемой статистики (UC-05) предусматривает построение диаграмм по выбранным пользователем параметрам. В текущей версии приложения данная страница реализована в виде заглушки.

Сценарии экспорта данных

Экспорт данных (UC-06) позволяет пользователю скачать полную копию базы данных в одном из трех форматов: JSON, CSV или XML. Экспортируемый файл содержит два раздела: все персоны со всеми

атрибутами и все связи между ними. Экспортированный файл может быть использован для резервного копирования или для загрузки в другой экземпляр приложения.

Вывод о преобладающих операциях

В данном приложении операции чтения значительно преобладают над операциями записи. Основные сценарии использования (просмотр графа, поиск персон, просмотр карточек, статистика) являются операциями чтения. Операции записи (добавление, редактирование и удаление персон, импорт данных) выполняются значительно реже. Графовая СУБД Neo4j оптимальна для данного профиля нагрузки, поскольку обеспечивает высокую производительность при обходе связей -- ключевой операции для визуализации родословного дерева.

2.1. Макет UI

Макет пользовательского интерфейса приложения доступен в вики репозитория проекта. Приложение содержит шесть основных страниц, доступных через верхнюю навигационную панель:

- 1) Главная -- общая статистика по династии и список последних добавленных персон.
- 2) Поиск -- форма фильтрации и таблица результатов поиска.
- 3) Граф -- интерактивная визуализация родословного дерева.
- 4) Статистика -- страница кастомизируемой статистики (заглушка).
- 5) Импорт/Экспорт -- загрузка и выгрузка данных в файлы.
- 6) Добавление персоны -- форма создания новой персоны с указанием связей.

3. Модель данных

3.1. Нереляционная модель данных (Neo4j)

В графовой СУБД Neo4j данные хранятся в виде узлов (nodes) и связей (relationships). Модель данных приложения включает один тип узла и один тип связи.

Графическое представление

Модель данных представляет собой ориентированный граф со следующей структурой:

Узлы с меткой Person содержат информацию о персонах династии. Связи типа RELATED_TO соединяют персоны и описывают родственные отношения (отец, мать, сын, дочь, супруг). Каждая связь является направленной и содержит тип как прямой, так и обратной связи.

Описание сущностей и типов данных

Узел Person:

Свойство	Тип данных	Описание
id	String (UUID)	Уникальный идентификатор
first_name	String	Имя
last_name	String	Фамилия
birth_year	Integer	Год рождения
death_year	Integer null	Год смерти
title	String null	Титул
country	String null	Страна
dynasty	String null	Династия
gender	String (M/F)	Пол
comment	String null	Комментарий
created_at	String (ISO 8601)	Дата создания записи
updated_at	String (ISO 8601)	Дата последнего изменения

Связь RELATED_TO:

Свойство	Тип данных	Описание
type	String	Тип связи (father, mother, son, daughter, spouse)
reverse_type	String	Обратный тип связи
start_date	String null	Дата начала (для брачных связей)
end_date	String null	Дата окончания (для брачных связей)

Оценка удельного объёма информации

Один узел Person занимает приблизительно 500 байт: UUID-идентификатор (36 байт), строковые поля имени, фамилии, титула, страны, династии (в среднем 300 байт), целочисленные поля годов (16 байт), временные метки (около 60 байт), служебные данные Neo4j (около 88 байт).

Одна связь RELATED_TO занимает приблизительно 100 байт: строковые поля типов связей (около 40 байт), необязательные даты (около 20 байт), служебные данные Neo4j (около 40 байт).

Для N персон со средним количеством R связей на персону общий объем данных составляет приблизительно $N * 500 + N * R * 100$ байт.

Текущий набор данных: 182 персоны, 766 связей. Оценка объёма: $182 * 500 + 766 * 100 = 91\,000 + 76\,600 = 167\,600$ байт (приблизительно 168 КБ).

При увеличении количества персон до 1000 при среднем количестве 4 связей на персону: $1000 * 500 + 4000 * 100 = 900\,000$ байт (приблизительно 900 КБ). Объём данных растёт линейно.

Запросы к модели данных

Ниже приведены ключевые запросы на языке Cypher, реализующие основные сценарии использования.

UC-01. Статистика династии:

```
MATCH (p:Person)
WITH count(p) AS total,
      avg(CASE WHEN p.birth_year IS NOT NULL
```

```

        AND p.death_year IS NOT NULL
        THEN p.death_year - p.birth_year END) AS avg_age,
        min(p.birth_year) AS start, max(p.death_year) AS end
OPTIONAL MATCH ()-[r:RELATED_TO]->()
RETURN total, avg_age, start, end, count(r) AS rels

```

UC-02. Поиск персон:

```

MATCH (p:Person)
WHERE toLower(p.first_name) CONTAINS toLower($name)
      AND p.birth_year >= $from AND p.birth_year <= $to
RETURN p ORDER BY p.last_name, p.first_name

```

UC-03. Карточка персоны с связями:

```

MATCH (p:Person {id: $id})-[r:RELATED_TO]->(rel:Person)
RETURN r.type AS rel_type, rel
UNION ALL
MATCH (rel:Person)-[r:RELATED_TO]->(p:Person {id: $id})
WHERE r.reverse_type IS NOT NULL
RETURN r.reverse_type AS rel_type, rel

```

UC-08. Создание персоны:

```

CREATE (p:Person {id: $id, first_name: $fn, ...})
// Для каждой связи:
MATCH (a:Person {id: $from}), (b:Person {id: $to})
CREATE (a)-[:RELATED_TO {type: $t, reverse_type: $rt}]->(b)

```

UC-10. Удаление персоны:

```

MATCH (p:Person {id: $id}) DETACH DELETE p

```

3.2. Аналог модели данных для SQL СУБД

Для сравнения приведена эквивалентная модель данных в реляционной СУБД.

Таблица persons:

Столбец	Тип	Ограничения
id	UUID	PRIMARY KEY
first_name	VARCHAR(255)	NOT NULL
last_name	VARCHAR(255)	NOT NULL
birth_year	INTEGER	
death_year	INTEGER	
title	VARCHAR(255)	
country	VARCHAR(255)	
dynasty	VARCHAR(255)	
gender	CHAR(1)	CHECK (gender IN ('M','F'))
comment	TEXT	
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Таблица relations:

Столбец	Тип	Ограничения
id	SERIAL	PRIMARY KEY
from_person_id	UUID	FK -> persons(id)
to_person_id	UUID	FK -> persons(id)
relation_type	VARCHAR(50)	NOT NULL
reverse_type	VARCHAR(50)	
start_date	VARCHAR(20)	
end_date	VARCHAR(20)	

Оценка удельного объёма информации

Строка в таблице persons занимает приблизительно 400-500 байт (аналогично узлу Neo4j). Строка в таблице relations занимает приблизительно 100-120 байт. С учетом индексов (PRIMARY KEY, FOREIGN KEY) накладные расходы составляют около 20-30%. Общий объём для 182 персон и 766 связей: приблизительно 200 КБ.

Запросы к реляционной модели данных

Ниже приведены эквивалентные SQL-запросы, реализующие основные сценарии использования. Полная версия запросов доступна на вики-странице репозитория проекта.

UC-01. Статистика династии:

```
SELECT COUNT(*) AS total,  
       AVG(death_year - birth_year) AS avg_age,  
       MIN(birth_year) AS dynasty_start,  
       MAX(death_year) AS dynasty_end  
FROM persons  
WHERE birth_year IS NOT NULL  
       AND death_year IS NOT NULL;
```

UC-02. Поиск персон:

```
SELECT * FROM persons  
WHERE LOWER(first_name) LIKE LOWER('%' || :name || '%')  
       AND birth_year >= :from AND birth_year <= :to  
ORDER BY last_name, first_name;
```

UC-03. Карточка персоны с связями:

```
SELECT r.relation_type, p2.*  
FROM relations r  
JOIN persons p2 ON r.to_person_id = p2.id  
WHERE r.from_person_id = :id  
UNION ALL  
SELECT r.reverse_type, p2.*  
FROM relations r  
JOIN persons p2 ON r.from_person_id = p2.id  
WHERE r.to_person_id = :id;
```

UC-08. Создание персоны:

```
INSERT INTO persons (id, first_name, last_name, ...)  
VALUES (:id, :fn, :ln, ...);  
-- Для каждой связи:  
INSERT INTO relations (from_person_id, to_person_id,  
                       relation_type, reverse_type)
```

```
VALUES (:from, :to, :type, :rev_type);
```

UC-10. Удаление персоны:

```
DELETE FROM relations
WHERE from_person_id = :id OR to_person_id = :id;
DELETE FROM persons WHERE id = :id;
```

3.3. Сравнение моделей

Удельный объём информации

При небольшом объёме данных (сотни персон) объём хранимых данных в обеих моделях сопоставим. Neo4j хранит связи как объекты первого класса с указателями, что добавляет небольшие накладные расходы на каждую связь, но устраняет необходимость в индексах по внешним ключам. SQL-модель требует дополнительных индексов на foreign key для обеспечения производительности JOIN-операций. При прочих равных условиях разница в объёме не превышает 10-15%.

Сравнение запросов по сценариям использования

Сценарий	Neo4j: кол-во запросов	SQL: кол-во запросов	Neo4j: сложность	SQL: сложность
UC-02 Поиск	1	1	Средняя	Средняя
UC-03 Карточка	1 (UNION)	1-2 (JOIN)	Средняя	Высокая
UC-04 Граф	2	3	Низкая	Средняя
Обход на глубину N	1	1 (рекурс. CTE)	O(N) хопов	O(N) JOIN
UC-08 Создание	1 + R	1 + R	Низкая	Низкая
UC-10 Удаление	1 (DETACH)	2-3 (каскад.)	Низкая	Средняя

Для простых операций поиска (UC-02) обе модели показывают сопоставимую производительность. Основные преимущества Neo4j проявляются при операциях, связанных с обходом связей:

При просмотре карточки персоны (UC-03) Neo4j выполняет обход связей напрямую через указатели, в то время как SQL требует JOIN-операции с таблицей relations и дополнительный JOIN для получения имён связанных персон.

При построении полного графа (UC-04) Neo4j возвращает все связи одним запросом, SQL требует соединения трёх таблиц.

При поиске предков или потомков на произвольную глубину Neo4j выполняет обход за $O(N)$ хопов с константным временем на каждый переход. SQL требует рекурсивного CTE, время выполнения которого растет с глубиной и объёмом таблицы связей.

Вывод

Для задачи хранения и визуализации генеалогического дерева графовая СУБД Neo4j является более подходящим выбором по сравнению с реляционной СУБД. Генеалогические данные имеют внутренне присущую графовую структуру, а ключевые операции приложения (визуализация дерева, навигация по связям, просмотр карточек с родственниками) сводятся к обходу графа. Neo4j обеспечивает обход связей за константное время на каждый переход, тогда как SQL требует JOIN-операций с растущей стоимостью при увеличении глубины обхода. Запросы на языке Cypher для графовых паттернов значительно более читаемы и компактны.

4. Разработанное приложение

4.1. Краткое описание

Приложение реализовано в виде трехконтейнерной архитектуры, управляемой через Docker Compose:

1) db -- контейнер с графовой СУБД Neo4j 5.18 Community Edition. Хранит все данные о персонах и связях между ними. Данные персистируются через Docker volume.

2) backend -- контейнер с серверной частью на Python 3.12 и фреймворке FastAPI. Предоставляет REST API для всех операций: CRUD-операции над персонами, поиск, статистика, импорт/экспорт. Взаимодействует с Neo4j через официальный Python-драйвер.

3) frontend -- контейнер с клиентской частью на React 19 и Vite 8. Реализует одностраничное приложение (SPA) с маршрутизацией через react-router-dom. Граф родословного дерева отрисовывается с помощью Canvas API.

При первом запуске backend автоматически заполняет базу данных начальным набором из 182 персон и 766 связей, содержащим исторические данные о династии Романовых.

4.2. Используемые технологии

Компонент	Технологии
Backend	Python 3.12, FastAPI 0.111, Uvicorn 0.29, neo4j 5.26 (драйвер), Pydantic 2.7, pydantic-settings 2.2
Frontend	React 19, Vite 8, react-router-dom 7, Canvas API (визуализация графа)
База данных	Neo4j 5.18 Community Edition
Инфраструктура	Docker, Docker Compose
CI/CD	GitHub Actions (7 workflow-проверок)

4.3. Снимки экрана приложения

Описание экранов приложения со снимками представлено в Приложении Б (Инструкция для пользователя). Приложение содержит шесть основных страниц: главная страница со статистикой, поиск персон, интерактивный граф родословного дерева, статистика, импорт/экспорт данных и форма добавления персоны. Все страницы связаны общей навигационной панелью.

5. Выводы

5.1. Достигнутые результаты

В ходе выполнения индивидуального домашнего задания разработано веб-приложение для хранения, визуализации и анализа генеалогического дерева династии Романовых с использованием графовой СУБД Neo4j.

Реализовано 9 из 10 сценариев использования: UC-01 (главная страница со статистикой), UC-02 (поиск и фильтрация персон), UC-03 (карточка персоны с родственными связями), UC-04 (интерактивный граф родословного дерева), UC-06 (экспорт данных в JSON, CSV, XML), UC-07 (импорт данных из файла), UC-08 (добавление персоны), UC-09 (редактирование персоны), UC-10 (удаление персоны). Сценарий UC-05 (кастомизируемая статистика) реализован в виде страницы-заглушки.

Приложение успешно хранит и обрабатывает 182 исторических персоны с 766 родственными связями. Визуализация графа поддерживает масштабирование, перемещение и интерактивный просмотр карточек персон.

5.2. Недостатки и пути для улучшения

К основным недостаткам текущей реализации относятся:

- 1) Сценарий UC-05 (кастомизируемая статистика с построением диаграмм) не полностью реализован.
- 2) Отсутствует серверная пагинация в результатах поиска, что может привести к проблемам производительности при большом объёме данных.
- 3) Отсутствует механизм аутентификации и авторизации пользователей.
- 4) Визуализация графа реализована на Canvas API вручную; использование специализированных библиотек (vis.js, d3.js) позволило бы улучшить алгоритм раскладки и интерактивность.

5.3. Будущее развитие решения

Для дальнейшего развития приложения предлагаются следующие направления:

- 1) Полная реализация страницы статистики (UC-05) с интерактивными диаграммами и настраиваемыми осями.
- 2) Добавление серверной пагинации во все таблицы.
- 3) Реализация системы аутентификации для разграничения доступа.
- 4) Улучшение алгоритма раскладки графа и переход на специализированную библиотеку визуализации.
- 5) Добавление расширенной аналитики: временные шкалы, генеалогические калькуляторы, поиск общих предков.

Приложение А. Документация по сборке и развертыванию

Предварительные требования

Для развертывания приложения необходимо наличие установленных Docker и Docker Compose.

Порядок развертывания

1. Клонировать репозиторий:

```
git clone https://github.com/moevm/nsql1h26-tree.git
```

2. Перейти в директорию проекта:

```
cd nsql1h26-tree
```

3. Запустить все контейнеры:

```
docker compose up --build
```

4. Открыть приложение в браузере по адресу: <http://localhost:5173>

5. REST API доступен по адресу: <http://localhost:8000>

6. Браузер Neo4j доступен по адресу: <http://localhost:7474>

Переменные окружения

Файл `.env` в корне репозитория содержит конфигурацию подключения к базе данных:

```
NEO4J_URI=bolt://db:7687
NEO4J_USER=neo4j
NEO4J_PASSWORD=myname1234
```

Остановка приложения

Для остановки всех контейнеров выполните:

```
docker compose down
```

Для остановки с удалением данных:

```
docker compose down -v
```

Приложение Б. Инструкция для пользователя

Главная страница

При открытии приложения отображается главная страница (Рис. 1) с общей статистикой: количество персон в базе, средний возраст, количество лет в истории. Ниже приводится список последних добавленных персон. Клик на персону открывает её карточку.

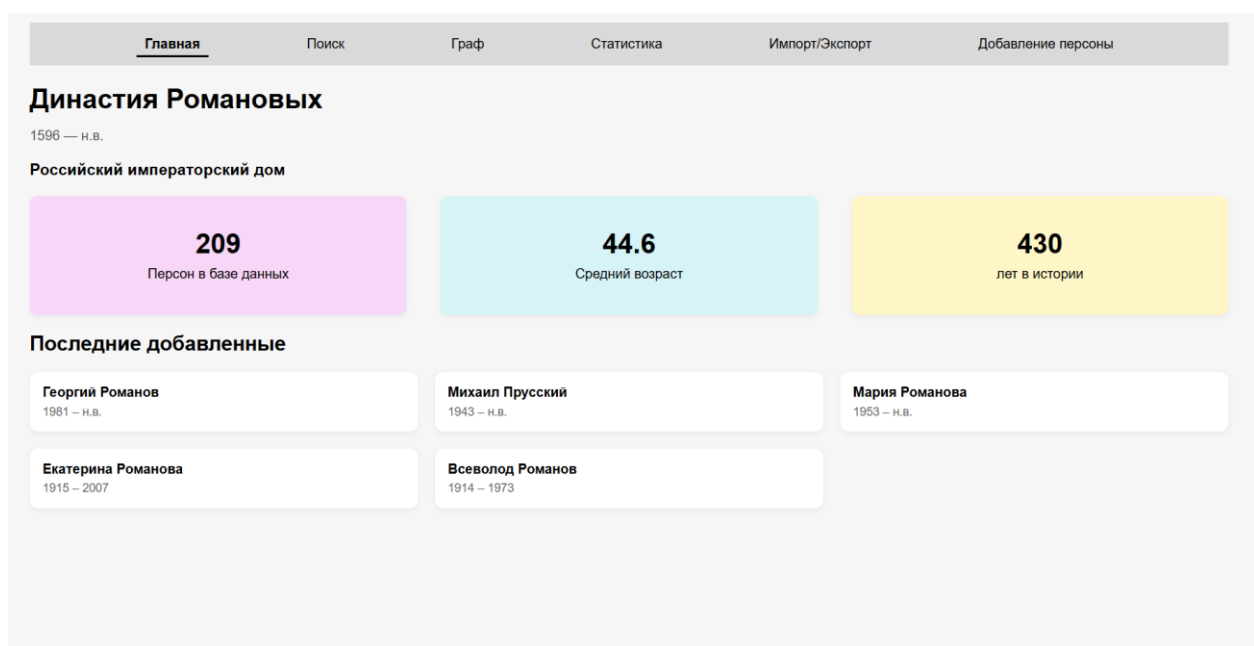


Рисунок 1. Главный экран

Поиск

Страница поиска (Рис. 2) позволяет найти персон по следующим критериям: имя, фамилия, титул (поиск по подстроке), пол, диапазон года рождения и года смерти. Заполните нужные фильтры и нажмите “Найти”. Результаты отобразятся в таблице. Клик по строке открывает карточку персоны.

Главная

Поиск

Граф

Статистика

Импорт/Экспорт

Добавление персоны

Поиск персон

Имя

Романов

Император

Пол

Год рождения от

до

Год смерти от

до

Найти

Результаты

Александр I	Романов	Император	1777 - 1825	M
Александр II	Романов	Император	1818 - 1881	M
Александр III	Романов	Император	1845 - 1894	M
Иоанн VI	Романов	Император	1740 - 1764	M
Кирилл I	Романов	Император	1876 - 1938	M
Николай I	Романов	Император	1796 - 1855	M
Николай II	Романов	Император	1868 - 1918	M
Павел I	Романов	Император	1754 - 1801	M
Пётр I	Романов	Царь/Император	1672 - 1725	M
Пётр II	Романов	Император	1715 - 1730	M

Рисунок 2. Страница поиска

Граф дерева

Страница графа отображает всё родословное дерево в виде интерактивной визуализации (Рис. 3). Узлы представляют персон, линии -- связи между ними. Для навигации используйте: колесо мыши для масштабирования, перетаскивание для перемещения, кнопки +/- и кнопку сброса вида. Клик на узел открывает карточку персоны.

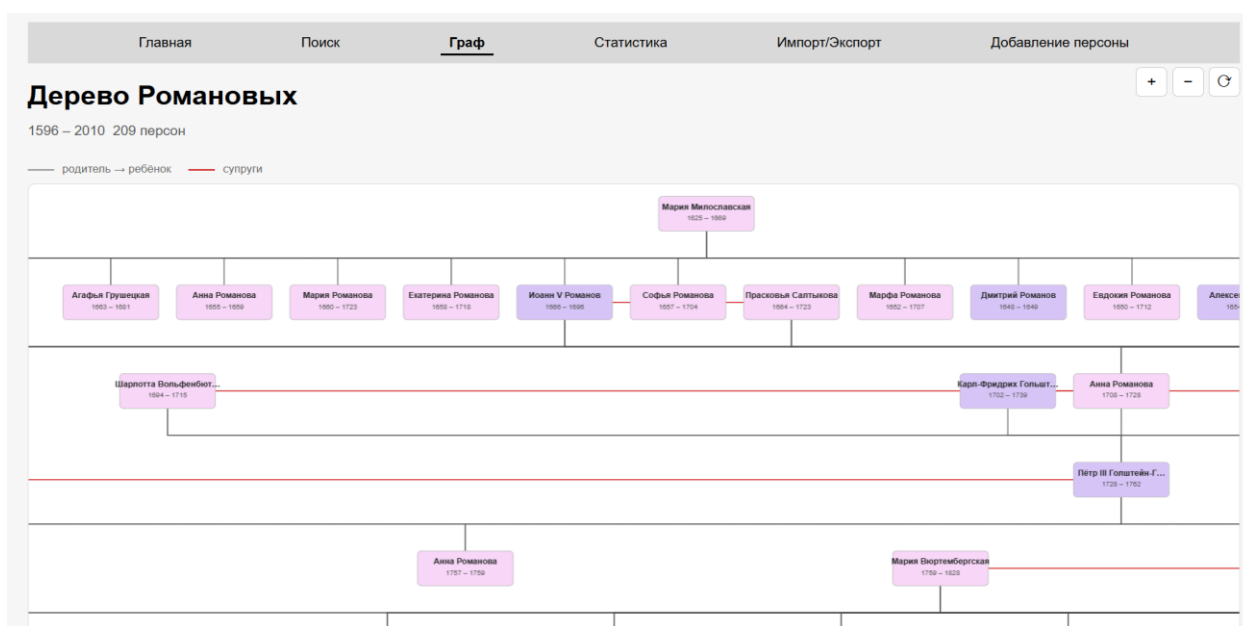


Рисунок 3. Граф дерева

Карточка персоны

Модальное окно карточки показывает полную информацию: имя, годы жизни, титул, страну, династию (Рис. 4). В таблице родственных связей перечислены все связанные персоны с указанием типа связи. Клик на имя родственника переключает карточку на него. Из карточки доступны кнопки “Редактировать” и “Удалить”.

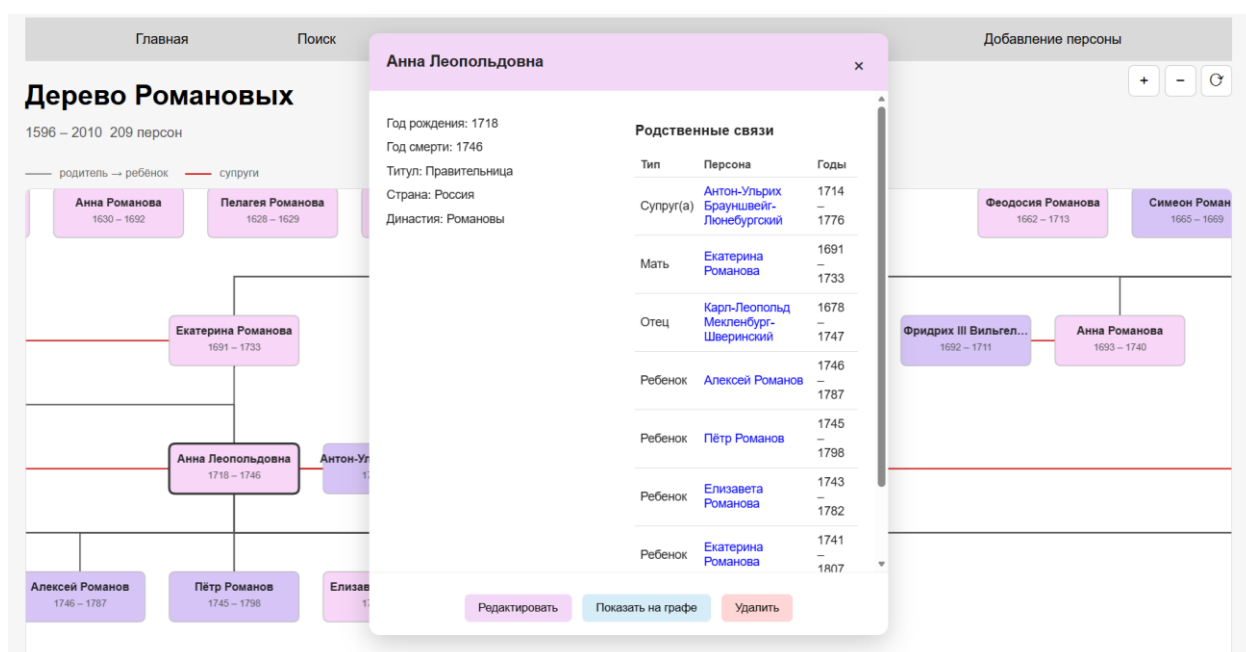


Рисунок 4. Карточка персоны

Импорт и экспорт

На странице “Импорт/Экспорт” доступны две операции (Рис. 5). Экспорт: выберите формат (JSON, CSV, XML) и нажмите “Экспортировать” - файл скачается в браузер. Импорт: выберите файл и нажмите “Импортировать” -- текущая база данных будет заменена данными из файла.

Рисунок 5. Импорт/Экспорт

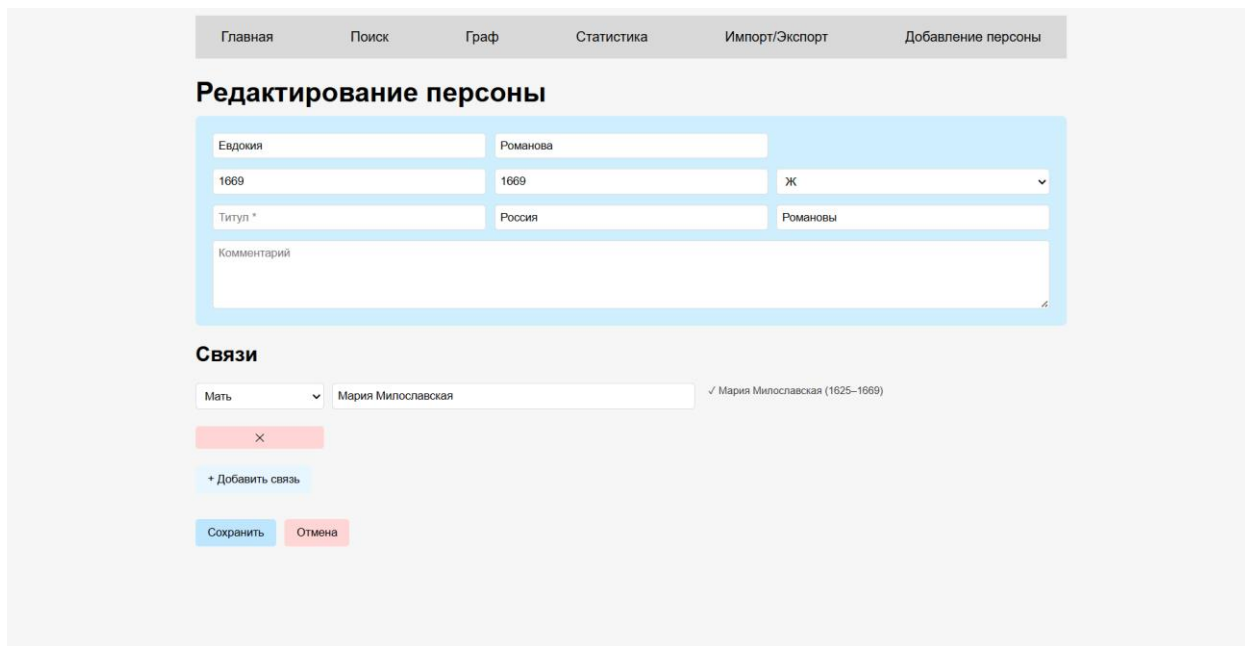
Добавление персоны

На странице “Добавление персоны” (Рис. 6) заполните обязательные поля (имя, фамилия, год рождения, титул, страна, династия, пол) и необязательные (год смерти, комментарий). Для добавления связей нажмите “+ Добавить связь”, выберите тип связи и найдите персону через поле поиска. Нажмите “Сохранить” для создания записи.

Рисунок 6. Добавление персоны

Редактирование персоны

На странице редактирования форма предзаполнена текущими данными персоны (Рис. 7). Измените необходимые поля, добавьте или удалите связи и нажмите “Сохранить”.



Главная Поиск Граф Статистика Импорт/Экспорт Добавление персоны

Редактирование персоны

Евдокия	Романова	
1669	1669	Ж
Титул *	Россия	Романовы
Комментарий		

Связи

Мать Мария Милославская ✓ Мария Милославская (1625–1669)

X

+ Добавить связь

Сохранить Отмена

Рисунок 7. Редактирование персоны

Литература

1. Neo4j Documentation [Электронный ресурс]. -- Режим доступа:
<https://neo4j.com/docs/>
2. FastAPI Documentation [Электронный ресурс]. -- Режим доступа:
<https://fastapi.tiangolo.com/>
3. React Documentation [Электронный ресурс]. -- Режим доступа:
<https://react.dev/>
4. Репозиторий проекта [Электронный ресурс]. -- Режим доступа:
<https://github.com/moevm/nsql1h26-tree>
5. Docker Documentation [Электронный ресурс]. -- Режим доступа:
<https://docs.docker.com/>
6. Vite Documentation [Электронный ресурс]. -- Режим доступа:
<https://vite.dev/>
7. Cypher Query Language [Электронный ресурс]. -- Режим доступа:
<https://neo4j.com/docs/cypher-manual/current/>